Information Processing Letters 109 (2009) 454–459

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Improving multikey Quicksort for sorting strings with many equal elements ☆,☆☆

Eunsang Kim, Kunsoo Park *

School of Computer Science and Engineering, Seoul National University, 599 Gwanak-ro, Gwanak-gu, Seoul, 151-742, South Korea

ARTICLE INFO

Article history:
Received 26 November 2008
Received in revised form 13 January 2009
Accepted 13 January 2009
Available online 19 January 2009
Communicated by K. Iwama

Keywords:
Algorithms
String sorting
Multikey Quicksort
Ternary partitioning

ABSTRACT

Bentley and Sedgewick proposed multikey Quicksort with 'split-end' partitioning for sorting strings. But it can be slow in case of many equal elements because it adopted 'split-end' partitioning that moves equal elements to the ends and swaps back to the middle. We present 'collect-center' partitioning to improve multikey Quicksort in that case. It moves equal elements to the middle directly like the 'Dutch National Flag Problem' partitioning approach and it uses two inner loops like Bentley and McIlroy's. In case of many equal elements such as DNA sequences, HTML files, and English texts, multikey Quicksort with 'collect-center' partitioning is faster than multikey Quicksort with 'split-end' partitioning.

1. Introduction

The string sorting problem is to sort a set of strings in lexicographically nondecreasing order. A well-known algorithm for this problem is multikey Quicksort due to Bentley and Sedgewick [2]. Like regular Quicksort [4,7], it partitions input strings into a set smaller than, a set equal to, and a set greater than a pivot; like radix sort [6], it moves onto the next character once the given character of the current input is known to be equal to the pivot.

Hoare [4] sketched a basic multikey Quicksort in a section on "Multi-word keys": "When it is known that a segment comprises all the items, and only those items, which have key values identical to a given value over their first n words, then, in partitioning this segment, comparison is made of the $(n+1)$th word of the keys." Bentley and McIlroy [1] implemented a ternary Quicksort that employs 'split-end' partitioning, which moves equal elements to the ends and swaps them back to the middle. Bentley and Sedgewick [2] presented multikey Quicksort for strings, adopting 'multi-word keys' and 'split-end' partitioning.

In case of many equal elements, however, Bentley and Sedgewick's multikey Quicksort needs a lot of operations to swap equal elements to the ends and back to the middle because it adopted 'split-end' partitioning. In this paper, we improve multikey Quicksort in such a case. First, we improve the operation of swapping equal elements from the ends back to the middle in 'split-end' partitioning. Then we improve multikey Quicksort by using another partitioning method called 'collect-center' partitioning, instead of 'split-end' partitioning. This partitioning blends the 'Dutch National Flag Problem' (DNF) ternary partitioning [3] and Bentley and McIlroy's. Like the DNF partitioning, it moves equal elements to the middle directly; like Bentley and McIlroy's, it uses two inner loops, one of which halts on a greater element and the other halts on a smaller element. In case of many equal elements, multikey Quicksort with our partitioning method is faster than multikey Quicksort with 'split-end' partitioning because ours moves equal elements to the middle directly, not wasting time swapping equal elements to the ends and back to the middle.

Section 2 reviews multikey Quicksort, the DNF partitioning, and 'split-end' partitioning. An improved 'split-

☆ A preliminary version of this paper was presented at the 19th International Workshop on Combinatorial Algorithms (IWOCA 2008).

☆☆ This work was supported by Korea Research Council of Fundamental Science and Technology.

* Corresponding author.
E-mail addresses: eskim@theory.snu.ac.kr (E. Kim), kpark@theory.snu.ac.kr (K. Park).

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2009.01.007

end' partitioning and 'collect-center' partitioning are presented in Sections 3 and 4, respectively. Section 5 shows experimental results with four partitioning methods. We conclude in Section 6.

2. Preliminaries

2.1. Multikey Quicksort

We first consider multikey Quicksort. Its input is an array S of n pointers to strings and it sorts strings in lexicographically nondecreasing order. Like Quicksort, it partitions S to sub-arrays smaller than, equal to, and greater than a pivot; like radix sort, it moves onto the next character to partition the sub-array equal to a pivot. Then it sorts three sub-arrays recursively.

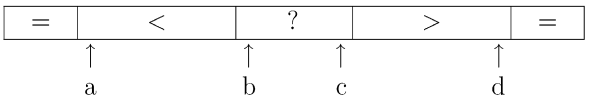
This recursive pseudo-code sorts the array S of n strings that are known to be identical in the first $d-1$ characters. It is initially called as sort$(S, n, 1)$. $S_<$, $S_=$, and $S_>$ mean sub-arrays smaller than, equal to, and greater than a pivot, respectively.

```
sort(S, n, d)
  if n ⩽ 1 return;
  choose a pivot v;
  partition S around v on dth character
    to form sub-arrays S<, S=, S> of sizes n<, n=, n>;
  sort(S<, n<, d);
  sort(S=, n=, d + 1);
  sort(S>, n>, d);
```

Choosing a pivot and partitioning S around the pivot can be implemented in many ways. In order to improve multikey Quicksort, we focus on the partitioning method of multikey Quicksort in this paper.

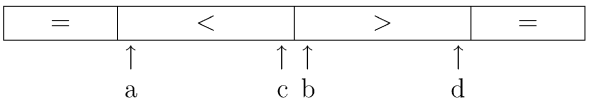
2.2. Bentley and McIlroy's 'split-end' partitioning

Bentley and Sedgewick adopted 'split-end' partitioning as the partitioning method of multikey Quicksort. 'Split-end' partitioning uses the following invariant.



Indices b and c point to the left and right ends of the array of unchecked elements, respectively. An index a points to the beginning of the array of smaller elements, and d points to the end of the array of greater elements.

As index b moves to the right, it scans over smaller elements, swaps equal elements to the element pointed to by a, and stops at a greater element. Symmetrically it moves index c to the left, scans over greater elements, swaps equal elements to the element pointed to by d, and stops at a smaller element. (That is, it swaps elements every time b and c meet equal elements.) Then, it swaps elements pointed to by b and c, increases pointer b, decreases pointer c, and continues until b and c cross. At the end, it reaches the following state.

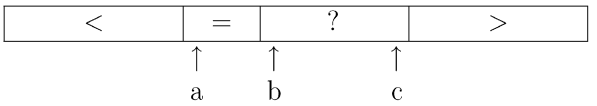


Finally, it swaps equal elements from the ends back to the middle.

This partitioning is usually efficient. In case of many equal elements, however, it wastes time swapping them to the ends and back to the middle.

2.3. 'Dutch National Flag Problem' partitioning

On the other hand, the DNF partitioning uses the following invariant.



Indices b and c point to the left and right ends of the array of unchecked elements, respectively. An index a points to the beginning of the array of equal elements.

As index b moves to the right, it scans over equal elements, swaps smaller elements to the element pointed to by a and greater elements to the element pointed to by c. In case of greater elements, it does not move index b because the swapped element is not checked. That is, it swaps elements every time b meets a smaller or greater element.

If the input such as integers has a few equal elements, the DNF partitioning can be slower than 'split-end' partitioning. In case of many equal elements, however, it just scans over equal elements and can be very fast.

3. An improved 'split-end' partitioning

In this section, we consider the operation of swapping equal elements from the ends back to the middle in 'split-end' partitioning. Let A be an array of equal elements at the left or right end and B be an array of smaller or greater elements in the middle. The operation of swapping equal elements from the ends back to the middle is the problem of swapping k elements in array A and k elements in array B, where the order of swapped elements does not need to be maintained.

For swapping equal elements from the ends back to the middle, Bentley and Sedgewick [2] use the usual swap operation k times as follows. $A[i]$ means the ith element in an array A.

```
for i ← 1 to k do
  t ← A[i]
  A[i] ← B[i]
  B[i] ← t
od
```

Now we define **movement** as an assignment operation and count movements of a swap operation. In the pseudo-code above, one swap operation consists of three movements. First, it copies an element in array A to a temporary space, t. Then, it copies an element in array B to array A. Finally, it copies the value in temporary space t to array B.

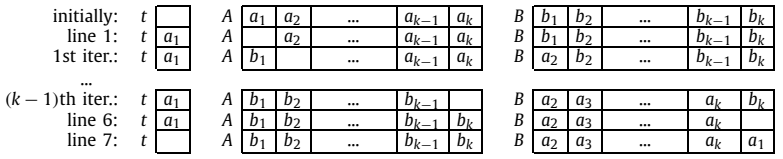


Fig. 1. Steps of batch-swap.

So, the swap operation of k elements consists of $3k$ movements.

However, we can decrease the number of movements as follows. We call this method **batch-swap**.

```
1  t ← A[1]
2  for i ← 1 to k − 1 do
3     A[i] ← B[i]
4     B[i] ← A[i + 1]
5  od
6  A[k] ← B[k]
7  B[k] ← t
```

It moves the first element in array A to the temporary space t, and it gets an empty space in array A (line 1). Next, it moves the first element in array B to that empty space, and it gets an empty space in array B again and repeats this step. After the *for* loop, we move the last element in array B to an empty space in array A, and move the element in t to an empty space in array B. Each step is shown in Fig. 1.

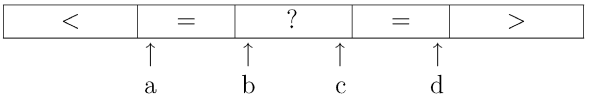
Now we count the number of movements of the batch-swap. Because there are $k-1$ iterations, we need $2(k-1)$ movements in the *for* loop. Since we need 3 movements outside the *for* loop, the batch-swap needs $2k+1$ movements for swapping k elements.

4. 'Collect-center' partitioning

In this section, we propose another partitioning method for multikey Quicksort. We also present the C program of multikey Quicksort adopting this method.

4.1. Details of 'collect-center' partitioning

The 'split-end' partitioning is not efficient in case of many equal elements. If we can partition equal elements directly to the middle instead of swapping them to the ends and back to the middle, this inefficiency can be removed. We call this method **'collect-center'** partitioning. This partitioning blends the DNF partitioning [3] and Bentley and McIlroy's [1]. Like DNF partitioning's approach, it moves equal elements to the middle directly. Like Bentley and McIlroy's approach, it uses a loop invariant that a partitioning is processed at both ends of an array. That is, 'collect-center' partitioning uses the following invariant.



Indices b and c point to the left and right ends of the array of unchecked elements, respectively. An index a points to the beginning of the left array of equal elements, and d points to the end of the right array of equal elements symmetrically.

As b moves to the right, it scans over equal elements, swaps smaller elements to the element pointed to by a, and stops at a greater element. Symmetrically it moves index c to the left, scans over equal elements, swaps greater elements to the element pointed to by d, and stops at a smaller element. Then, it swaps elements pointed to by b and c. But it does not increase and decrease these pointers because the element pointed to by b is a smaller element and the element pointed to by c is a greater element. So, it needs to swap the element pointed to by b for the element pointed to by a and swap the element pointed to by c for the element pointed to by d. After that, it moves those pointers and continues until b and c cross.

The above partitioning has an inefficiency that it needs to swap three times when pointers b and c are stopped. It first swaps elements pointed to by b and c, next swaps elements pointed to by a and b and elements pointed to by c and d. For removing this inefficiency, we apply the batch-swap approach to this case. In order to get the same result of 3 swap operations, a smaller element pointed to by c should be moved to the place pointed to by a and a greater element pointed to by b to the place pointed to by d, and equal elements pointed to by a and d should be moved to the places pointed to by b and c, respectively. We can do movements as follows to satisfy these conditions.

```
1  t ← element pointed to by b
2  move element pointed to by a to b
3  move element pointed to by c to a
4  move element pointed to by d to c
5  move t to d
```

The procedure above obtains the same result and it decreases 9 movements (3 swaps) to 5 movements. The 5 movements are shown in Fig. 2, where $a_=$ means the equal element pointed to by a, $b_>$ the greater element pointed to by b, etc. The reverse order which starts with moving the element pointed to by c to t obtains the same result, too.

The order of movements is important. If the order of movements is changed, the result may be wrong when pointers a and b point to the same position or pointers c and d do. For example, we assume that pointers a and b point to the same position.

```
1  t ← element pointed to by a
2  move element pointed to by c to a
3  move element pointed to by d to c
4  move element pointed to by b to d
5  move t to b
```

init.:	t		$<$	$a_=$	$=$	$b_>$	?	$c_<$	$=$	$d_=$	$>$
line 1:	t	$b_>$	$<$	$a_=$	$=$		?	$c_<$	$=$	$d_=$	$>$
line 2:	t	$b_>$	$<$		$=$	$a_=$	?	$c_<$	$=$	$d_=$	$>$
line 3:	t	$b_>$	$<$	$c_<$	$=$	$a_=$	?		$=$	$d_=$	$>$
line 4:	t	$b_>$	$<$	$c_<$	$=$	$a_=$	?	$d_=$	$=$		$>$
line 5:	t		$<$	$c_<$	$=$	$a_=$	?	$d_=$	$=$	$b_>$	$>$

Fig. 2. Steps of 5 movements instead of 3 swaps.

init.:	t		$<$	$a_>, b_>$	?	$c_<$	$=$	$d_=$	$>$
line 1:	t	$a_>$	$<$	$b_>$	?	$c_<$	$=$	$d_=$	$>$
line 2:	t	$a_>$	$<$	$b_<, c_<$	?		$=$	$d_=$	$>$
line 3:	t	$a_>$	$<$	$b_<, c_<$	?	$d_=$	$=$		$>$
line 4:	t	$a_>$	$<$	$c_<$	?	$d_=$	$=$	$b_<$	$>$
line 5:	t		$<$	$a_>$	?	$d_=$	$=$	$b_<$	$>$

Fig. 3. A wrong case of movements.

```
void ccsort(char **a, int n, int depth) {
    int r, v;
    char **pa, **pb, **pc, **pd, **pn, *t;
    ...    // pivot selection phase
    for (;;) {
        while (pb <= pc && (r = p2c(pb)-v) <= 0) {
            if (r < 0)  { swap(pa, pb); pa++; }
            pb++;
        }
        while (pb <= pc && (r = p2c(pc)-v) >= 0) {
            if (r > 0)  { swap(pc, pd); pd--; }
            pc--;
        }
        if (pb > pc) break;
        t = *pb;
        *pb = *pa;
        *pa = *pc;
        *pc = *pd;
        *pd = t;
        pb++;
        pc--;
        pa++;
        pd--;
    }
    pn = a + n;
    if ((r = pa-a) > 1)
        ccsort(a, r, depth);
    if (ptr2char(a + r) != 0)
        ccsort(a + r, pd+1-pa, depth+1);
    if ((r = pn-pd-1) > 1)
        ccsort(pn - r, r, depth);
}
```

Fig. 4. C program using our 'collect-center' partitioning for multikey Quicksort.

If we change the order of movements as above, it overwrites a greater element pointed to by b ($b_>$) with a smaller element pointed to by c ($c_<$) (line 2) and it copies an overwritten smaller element pointed to by b ($b_<$) to the position pointed to by d in the end (line 4). The movements are shown in Fig. 3, and the result is wrong.

4.2. C program of multikey Quicksort with 'collect-center' partitioning

We assume that readers are familiar with the C programming language and explain the details of 'collect-center' partitioning with C program. The C program of multikey Quicksort with 'collect-center' partitioning is shown in Fig. 4.

The sort function is similar to the code of Bentley and Sedgewick [2], i.e., multikey Quicksort with 'split-end' partitioning. Its input is the array a of n pointers to strings. It sorts strings in lexicographically nondecreasing order with the *depth*th characters of strings. In the code above, the pivot selection phase is omitted, which is the same as the code of Bentley and Sedgewick [2] choosing the median of three elements (and on larger arrays, the median of three medians of three) for the pivot v described by Sedgewick [8]. $p2c(i)$ macro (for 'pointer to character') accesses the *depth*th character of string $a[i]$. Pointers *pa*, *pb*, *pc*, and *pd* mean pointers a, b, c, and d in Fig. 2, respectively. After partitioning, it sorts strings of three parts recursively. The most important difference between the two is just which elements pointers b and c scan over. In the code of Bentley

Table 1
Input data set used for comparison.

File	Description	Size (byte)	Number of strings
ran	A collection of strings of 1000 random characters	50,050,000	50,000
kordic	A collection of Korean dictionary words encoded by EUC-KR	5,837,700	631,026
words	A set of words in /usr/share/dict/words file of Linux	4,951,020	479,625
fasta	*Mus Musculus* fragments set of DNA sequences from NCBI	224,803,104	277,490
circ	Data set of library call numbers used in the DIMACS Implementation Challenge from http://theory.stanford.edu/~csilvers/libdata/. We extract the set of unique keys from the file.	1,931,910	85,595
bible	A set of verses of the 'New International Version' Bible	4,137,735	31,102
ran_ab	A collection of strings in which the letters 'a' and 'b' are repeated 1000 times randomly	50,050,000	50,000
html_kr	A collection of about 1000 html files from servers in Korea	47,690,332	823,762
html_cnn	A collection of about 1000 html files from www.cnn.com	60,544,921	1,036,505
aaa2000	All the suffixes of the string that consists of 2000 'a's	125,750	2000

Table 2
Sorting times in seconds. Lowest time is in bold face. These are in order of the average of lcp.

File	Avg. length of strings	Avg. of lcp	Ratio of equal elements	-O0 option (no optimization)			
				split-end	batch-swap	collect-center	DNF
ran	1000.0	1.3	0.0993	0.0477	**0.0475**	0.0519	0.0518
kordic	8.3	4.7	0.2752	0.4310	**0.4296**	0.4473	0.4402
words	9.3	6.3	0.3532	0.3439	**0.3423**	0.3497	0.3465
fasta	809.1	11.7	0.5828	0.5055	0.5037	0.4874	**0.4852**
circ	21.6	14.0	0.6196	0.0584	0.0582	0.0544	**0.0534**
bible	132.0	14.2	0.6160	0.0341	0.0339	**0.0324**	0.0325
ran_ab	1000.0	14.5	0.6627	0.0717	0.0710	**0.0670**	0.0672
html_kr	57.4	47.3	0.9047	2.3092	2.3029	**2.0716**	2.0839
html_cnn	56.9	48.4	0.9161	3.2657	3.2558	**3.0174**	3.0300
aaa2000	1000.5	1000.0	0.9990	0.0608	0.0602	0.0482	**0.0478**

and Sedgewick [2], pointers b and c scan over smaller and greater elements, respectively. But in this code, pointers b and c scan over equal elements. And the code of Bentley and Sedgewick [2] just swaps elements pointed to by b and c when pointers b and c stop, but our code does slightly complex assignment operations adopting the batch-swap approach as we described in the preceding subsection. Finally, the code of Bentley and Sedgewick swaps equal elements from the ends back to the middle, but our code does not need this step because equal elements are already in the middle.

5. Experimental results

We had experiments with four methods for multikey Quicksort, which are the DNF partitioning [3], Bentley and Sedgewick's 'split-end' partitioning [2], the improved 'split-end' partitioning using the batch-swap, and 'collect-center' partitioning. We downloaded the code of Bentley and Sedgewick at http://www.cs.princeton.edu/~rs/strings. All the methods had been implemented in C, compiled with gcc 3.2, with no optimization. We have the experiments in 2.4 GHz Xeon with 2 GB RAM, running GNU/Linux 2.4.20-28.

We measured the sorting time of multikey Quicksort with various inputs. For each input, we experimented 100 times repeatedly with shuffling pointers to strings in the array at each time, because multikey Quicksort can have different performances according to the order of input strings. The reported average times were measured with *gettimeofday()* function. As inputs, we used a set of large files, listed in Table 1. We used various inputs to compare the partitionings for different kinds of ratio of equal elements.

Table 2 shows sorting time of multikey Quicksort with four partitionings in seconds, listed in order of the average of lcp, *longest common prefix*, for adjacent strings in sorted order. The ratio of equal elements is computed by the following equation:

$$\frac{\sum_i \frac{e_i}{n_i} \cdot n_i}{\sum_i n_i} = \frac{\sum_i e_i}{\sum_i n_i},$$

where e_i is the number of equal elements and n_i is the input array size in the ith recursive partitioning operation. The value $\frac{e_i}{n_i}$ means the percentage of equal elements in the ith partitioning. This value is averaged with weight n_i over all recursive partitioning operations. That is, the ratio means the average percentage of equal elements in a recursive partitioning.

For example, consider sorting strings of *aaa2000*. The input string set of *aaa2000* is $\{a^i \mid 1 \leqslant i \leqslant 2000\}$. In the first recursive partitioning, multikey Quicksort checks the first characters of input strings. The first characters of all input strings are the letter 'a', and thus e_1 and n_1 are 2000. In the second recursive partitioning, it checks the second characters. The second characters of input strings are the letter 'a' except for input string a, and thus e_2 is 1999 and n_2 is 2000. Repeating these, in the last partitioning e_{2000} is 1 and n_{2000} is 2. Hence, the ratio of equal elements for *aaa2000* is $\frac{2000+1999+\cdots+1}{2000+2000+\cdots+2} = 0.9990$.

Since this ratio of equal elements varies with the initial order of input strings, we computed the average of all the ratios when we experimented 100 times repeatedly and presented it in Table 2. The order of the ratio is similar to

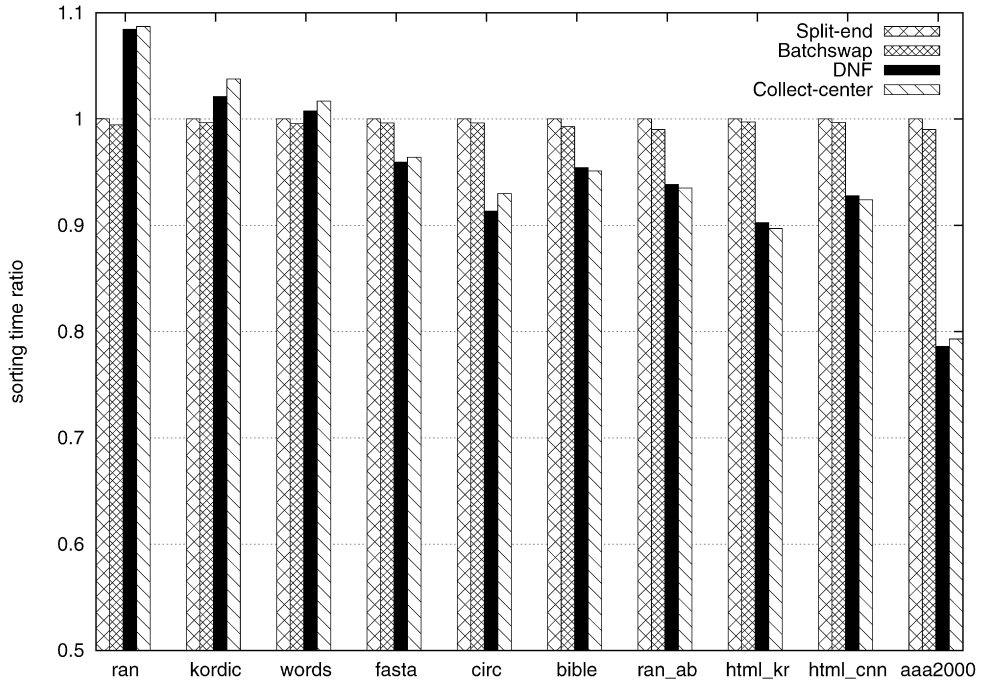


Fig. 5. The sorting time ratio of 'collect-center' partitioning, the batch-swap, and the DNF partitioning to 'split-end' partitioning. Methods are compiled with no optimization.

the order of the average of lcp. These two parameters represent how many equal elements are in input data while partitioning.

Fig. 5 shows the time ratio of 'collect-center' partitioning, the batch-swap, and the DNF partitioning to 'split-end' partitioning with no optimization option. The result shows that the batch-swap is slightly faster than 'split-end' partitioning in all the cases, and 'collect-center' partitioning and the DNF partitioning are faster than 'split-end' partitioning except for *ran*, *kordic*, and *words*.[1] The ratios of equal elements of these three inputs are under 0.4 and the averages of lcp are under 7.

6. Conclusion

Bentley and Sedgewick proposed multikey Quicksort with 'split-end' partitioning for sorting strings in [2]. But it can be slow in case of many equal elements, because 'split-end' partitioning moves equal elements to the ends and swaps back to the middle. There are many equal elements in data with a small alphabet size like DNA sequences or in data with some dominant prefixes like HTML files. These data tend to have the high average of lcp and the high ratio of equal elements. We improved multikey Quicksort by adopting the DNF partitioning and 'collect-center' partitioning in that case.

References

[1] J.L. Bentley, M.D. McIlroy, Engineering a sort function, Software: Practice and Experience 23 (1) (1993) 1249–1265.
[2] J.L. Bentley, R. Sedgewick, Fast algorithms for sorting and searching strings, in: Proceedings of the 8th Annual ACM–SIAM Symposium on Discrete Algorithms, New Orleans, January 1997, pp. 360–369.
[3] E.W. Dijkstra, A Discipline of Programming, Prentice-Hall, Englewood Cliffs, NJ, 1976.
[4] C.A.R. Hoare, Quicksort, Computer Journal 5 (1) (April 1962) 10–15.
[5] E. Kim, K. Park, Improving multikey Quicksort for sorting strings, in: Proceedings of the 19th International Workshop on Combinatorial Algorithms, 2008, pp. 89–99.
[6] P.M. McIlroy, K. Bostic, M.D. McIlroy, Engineering radix sort, Computing Systems 6 (1) (1993) 5–27.
[7] R. Sedgewick, Quicksort with equal keys, SIAM Journal on Computing 6 (1977) 240–267.
[8] R. Sedgewick, Implementing Quicksort programs, Communications of the ACM 21 (10) (October 1978) 847–857.

[1] In [5], it was reported that 'collect-center' partitioning is always faster than 'split-end' partitioning when compiled with -O3 option for maximum optimization. As a result of analyzing Assembly codes with maximum optimization, we found that three of pointers *a*, *b*, *c*, and *d* used in 'collect-center' partitioning are stored in registers and fewer pointers of other partitioning methods are stored in registers. Thus, other methods spent more time to copy the pointers in a memory stack to registers.